C. Array

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Chris the Rabbit has been interested in arrays ever since he was a child. At the moment he is researching arrays with the length of n, containing only integers from 1 to n. He is not good at math, that's why some simple things drive him crazy. For example, yesterday he grew keen on counting how many different beautiful arrays there are. Chris thinks that an array is beautiful if it meets one of the two conditions:

- each elements, starting from the second one, is no more than the preceding one
- each element, starting from the second one, is no less than the preceding one

Having got absolutely mad at himself and at math, Chris came to Stewie and Brian to ask them for help. However, they only laughed at him and said that the answer is too simple and not interesting. Help Chris the Rabbit to find the answer at last.

Input

The single line contains an integer n which is the size of the array ($1 \le n \le 10^5$).

Output

You must print the answer on a single line. As it can be rather long, you should print it modulo 1000000007.

Examples

input Copy

```
2
```

output Copy

```
4
```

input Copy

```
3
```

output Copy

```
17
```

Count arrays of length n:

- Values in [1, n]
- Array is non-decreasing OR non-increasing
- Answer modulo 1e9+7

Focus on Non-Decreasing Arrays

Condition:

a1 ≤ a2 ≤ ... ≤ an

Let:

x1 = count of 1
x2 = count of 2
...
xn = count of n

Then:

x1 + x2 + ... + xn = n
xi ≥ 0

Once xi are fixed, the array is uniquely determined:

1 repeated x1 times,
2 repeated x2 times,
...
n repeated xn times

So:

Counting non-decreasing arrays
= counting non-negative integer solutions of the equation above

Apply Stars and Bars (Visual Idea)

- n stars → array positions
- n − 1 bars → separate values 1..n

Total symbols:

n stars + (n − 1) bars = 2n − 1

Choose positions for bars:

$$Non-decreasing\ arrays = \binom{2n-1}{n-1}$$

Non-Increasing Arrays

Non-increasing arrays are just the reverse of non-decreasing arrays.

$$Non-increasing\ arrays = \binom{2n-1}{n-1}$$

Fix Double Counting

Arrays counted twice:

[1,1,1,...], [2,2,2,...], ..., [n,n,n,...]

Number of constant arrays: n

Final Formula

$$\boxed{Answer = 2 \cdot \binom{2n-1}{n-1} - n}\ mod(10^9+7)$$

Code

```
#include <bits/stdc++.h>
using namespace std;

static const long long MOD = 1000000007;
static const int MAXN = 200000; // enough for 2*n

long long fact[MAXN + 1], invfact[MAXN + 1];

long long modpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++) {
        fact[i] = fact[i - 1] * i % MOD;
    }

    invfact[MAXN] = modpow(fact[MAXN], MOD - 2);
    for (int i = MAXN; i > 0; i--) {
        invfact[i - 1] = invfact[i] * i % MOD;
    }
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute();

    int n;
    cin >> n;

    long long non_decreasing = nCr(2 * n - 1, n - 1);
    long long ans = (2 * non_decreasing % MOD - n + MOD) % MOD;

    cout << ans << "\n";
    return 0;
}
```